

Google Kubernetes Engine (GKE)

Training

Module Notes, Hands-On Exercises, Core Concepts & Study Reference

David Grunwald · Architecting with Google Kubernetes Engine · August 2026

CONTENTS

- 1. Portfolio Summary — Exercises Completed
- 2. Google Cloud Foundations
- 3. Containers & the Kubernetes Object Model
- 4. Kubernetes Architecture & the Control Plane
- 5. Workload Controllers & Services
- 6. Storage, Namespaces & Resource Management
- 7. AI-Augmented GKE Operations (Gemini CLI & MCP)
- 8. Secure Agentic Execution (GKE Sandbox & gVisor)
- 9. Core Concept Review & Exam Questions
- 10. Appendix: Study URLs by Topic

1 Portfolio Summary — Exercises Completed

A crisp record of the hands-on work completed during this training, suitable as portfolio evidence.

EXERCISE	WHAT WAS ACCOMPLISHED
Autopilot cluster build	Stood up a GKE Autopilot cluster from an empty account on Windows/ PowerShell: created project, linked billing, enabled APIs, deployed and scaled an NGINX workload to three replicas across two nodes, exposed it to the internet via a LoadBalancer Service, verified the live page, then tore everything down with a confirmed zero-resource end state.
Cost-hygiene workflow	Practised end-to-end cost discipline: service-before-cluster teardown to release the load balancer, then verified no orphaned clusters or forwarding rules remained — a repeatable, budget-safe lab pattern.
AI-assisted troubleshooting	

EXERCISE	WHAT WAS ACCOMPLISHED
	Used Gemini CLI with the GKE MCP server to debug a broken deployment with full infrastructure awareness; compared manual vs. AI-assisted workflows and codified the fix into a reusable Agent Skill.
Secure agentic execution	Deployed a workload under the Kubernetes-sigs Agent Sandbox / GKE Sandbox to run untrusted agent code safely, using gVisor kernel-level isolation selected per-pod via RuntimeClass.
Documentation artifacts	Produced portfolio-ready deliverables from the work: a reproducible build guide, a Deployments/ReplicaSets training reference, and a Kubernetes High-Availability guide.

PORTFOLIO LABEL

Completed the *Architecting with Google Kubernetes Engine: Foundations* hands-on track (Aug 2026): provisioned and operated managed Kubernetes on GKE, exercised the declarative workload model end to end, and applied modern AI-assisted and secure-sandbox operations — all with cost-aware cluster hygiene.

2 Google Cloud Foundations

Cloud computing & the GCP footprint

Cloud computing delivers resources **as a service** on Google's hardware. GCP organizes its global footprint into **regions** (independent geographic areas) subdivided into **zones** (isolated failure domains). Spreading resources across zones and regions is the basis of availability.

Resource hierarchy

GCP organizes resources top-down: **Organization** → **Folders** → **Projects** → **Resources**. Folders group projects that share IAM policy and org-policy inheritance; the **project** is the base unit that actually holds resources and to which billing attaches. For a new application, the standard layout is a folder under the existing org with projects (often split by environment) inside it.

Billing & cost control

Billing is set up at the **project** level — a billing account links to one or more projects (a project with no linked account can use only free services). Three tools keep costs visible and safe:

TOOL	PURPOSE
Budgets & alerts	Set at billing-account or project level; notify at threshold percentages (e.g. 90% of a \$20,000 budget = \$18,000). A webhook can trigger automation such as shutting down resources.

TOOL	PURPOSE
Billing export to BigQuery	Streams detailed usage/cost data to a dataset for analysis or Data Studio visualization.
Reports	Console visual tool to monitor spend by project or service.

Separately, **quotas** limit resource *consumption* per project to prevent runaway usage from error or attack. Rate quotas reset periodically (e.g. GKE's default 3,000 API calls/project/min); allocation quotas cap totals. Quotas control usage, not dollars — budgets control cost.

EXAM DISTINCTION

To *manage cost* for a product, set **budgets and alerts** (they notify and can trigger automation). Linking a billing account merely *enables* charging; quotas cap *consumption*; neither is a cost-management control on its own.

Compute options & tools

CHOICE	WHEN IT FITS
GKE	Managed compute platform with native container support — maximum control over container config/deployment without operating the cluster yourself.
Cloud Run	Serverless containers; scales to zero; ideal for stateless, request-driven workloads.
Cloud Functions	Serverless event-driven functions.
Compute Engine	Raw VMs (IaaS); per-second billing and custom machine types give fine-grained cost control.
Cloud SDK (gcloud CLI)	Installed locally to script and automate against GCP (gcloud, gsutil, bq). Cloud Shell is the same tooling, browser-hosted.

3 Containers & the Kubernetes Object Model

Why containers

Containers deliver applications as **lightweight, standalone, resource-efficient, portable** packages. They solve three problems: isolating each app's dependencies, eliminating “works on my machine” drift between laptop and production, and avoiding the waste of packaging apps in full virtual machines. They share the host kernel and isolate at the process level, starting in seconds.

LINUX MECHANISM	ROLE
Union file systems	Layer an image into clean, minimal, cacheable read-only layers.

LINUX MECHANISM	ROLE
Namespaces	Control what a container can see (directory tree, network/IPs).
cgroups	Control how much a container can use (CPU, memory).

WRITABLE TOP LAYER

An app can only modify the single writable top layer; all lower layers are read-only. That top layer is **ephemeral** — when the container is deleted its contents are lost. This is why durable data must live outside the container.

The object model & declarative management

Every thing Kubernetes manages is an **object** — a persistent entity holding a **spec** (desired state you declare) and a **status** (current state reported by the control plane). Under **declarative management**, you declare the desired state and Kubernetes' **watch loop** continuously compares reality to that declaration and reconciles any difference, keeping it there despite failures.

Pods

The **Pod** is the smallest deployable object — not the container. Every running container lives in a Pod, which can hold one or more tightly-coupled containers that share networking and storage. Each Pod gets a unique IP; containers within a Pod share the network namespace and reach each other over `localhost` (127.0.0.1). To co-locate containers for minimum latency, place them in the **same Pod**. Note: bare Pods are **not self-healing** — that requires a controller.

4 Kubernetes Architecture & the Control Plane

A cluster is a set of computers (VMs in GKE). One coordinates — the **control plane** — and the others are **nodes** that run your workloads.

COMPONENT	ROLE
<code>kube-apiserver</code>	The single entry point. Accepts all view/change commands, authenticates, authorizes, validates, and manages admission control. Every component and every <code>kubectl</code> call talks through it.
<code>etcd</code>	The cluster datastore holding all state; needs an odd-numbered quorum for HA.
<code>kube-scheduler</code>	Decides which node each new Pod runs on.
<code>kube-controller-manager</code>	Runs reconciliation control loops driving actual state toward desired state.
<code>kubelet</code>	Kubernetes' agent on each node; starts/stops Pod containers and reports health.
<code>kube-proxy</code> / CNI	Maintain Pod and Service networking across the cluster.

COMPONENT	ROLE
CoreDNS (kube-dns)	In-cluster DNS for service discovery.

KUBECTL TALKS TO THE API SERVER

Every `kubectl` command is an HTTP request to **kube-apiserver**. The scheduler, controller manager, and kubelets never take commands directly from `kubectl` — everything routes through the API server.

Kubernetes vs. GKE

Kubernetes is the open-source orchestration platform; **GKE is Kubernetes as a managed service** — Google runs the control plane, and (in Autopilot) the nodes too. GKE also scales the cluster *as a whole* via cluster autoscaling, which open-source Kubernetes does not do on its own.

REGIONAL CLUSTER BILLING

A regional cluster replicates across **3 zones by default**, and the per-pool node count is created *in each zone*. Four nodes in the default pool × three zones = **12 billable Compute Engine VMs**. Control-plane machines aren't billed as VMs — they're covered indirectly by the cluster management fee.

5 Workload Controllers & Services

CONTROLLER	USE IT FOR
Deployment	Stateless apps: declares desired replicas and manages rollouts, rollbacks, and history by orchestrating ReplicaSets. Manage this, not Pods directly.
ReplicaSet	Guarantees N identical replica Pods via a reconciliation loop; created and managed for you by a Deployment.
StatefulSet	Stateful apps needing persistent storage — stable identity and a per-Pod PersistentVolume that reattaches on reschedule.
DaemonSet	Runs a Pod on every (or selected) node — node-level agents like logging/monitoring.

Deployments & the two-ReplicaSet rollout

To deploy multiple load-balanced copies, create a **Deployment manifest with a replica count** — not repeated Pod manifests. For a rolling upgrade, the Deployment creates a *second* ReplicaSet and shifts replicas from old to new (old drains 3 → 0 while new fills 0 → 3), retaining the old at zero for instant rollback. `maxUnavailable` and `maxSurge` govern the pace.

Services

A **Service** provides a stable, load-balancing network endpoint for a set of Pods, and lets you choose how they're exposed via `type` : `ClusterIP` (internal), `NodePort` (per-node port), or `LoadBalancer` (external cloud LB with a public IP). It hides Pod churn behind one durable virtual IP.

6 Storage, Namespaces & Resource Management

Persistent storage

Durable storage must be **remote to the Pod**: create Volumes on network-based storage (e.g. Persistent Disk, Filestore) provisioned as PersistentVolumes, claimed by PVCs, and mounted into the Pod. Because the data lives off-Pod, it survives Pod failure, deletion, or rescheduling to another node. Local node storage does *not* — it's lost on node failure or reschedule.

Namespaces & resource quotas

To keep production isolated from test/staging: configure separate **Namespaces** for each environment, then apply **ResourceQuotas to the test and staging Namespaces** to cap their CPU/memory so they cannot starve production. Quotaing production would limit the very workloads you want to prioritize. Labels are only metadata and cannot carry a quota; ResourceQuotas apply to Namespaces.

HA BUILDING BLOCKS

Multiple replicas + topology spread/anti-affinity + PodDisruptionBudgets + readiness/liveness probes + a Service, on a **regional** cluster, is the recipe for a workload that stays running through node and zone failures.

7 AI-Augmented GKE Operations (Gemini CLI & MCP)

Troubleshooting broken deployments traditionally means manual log-digging, `kubectl describe`, and YAML cross-referencing. General chatbots help with concepts but operate in a vacuum — blind to your codebase and live cluster state. This track closes that gap with progressively richer context.

TOOL	WHAT IT PROVIDES
Gemini CLI	A terminal-native interface to Gemini — pipe command output into the model and act without leaving the shell.
Model Context Protocol (MCP)	Open standard letting the model connect to real tools/data. The GKE MCP server lets Gemini CLI query your project's API, inspect cluster state, and act on your behalf.
Agent Skills	Packaged instructions/scripts/resources that codify org standards and automate repeatable workflows (e.g. a reusable troubleshooting skill that improves time-to-resolution).

TOOL	WHAT IT PROVIDES
kubect <code>l</code> -ai	AI-assisted kubect <code>l</code> workflows.

Platform engineering frames the goal: build internal tooling and a standardized “golden path” so developers deploy safely and quickly while the platform team keeps control of governance and cost. The mental model from class: *AI model = engine, GKE = chassis, sandboxes = safety, data pipelines = fuel*.

8 Secure Agentic Execution (GKE Sandbox & gVisor)

Running untrusted or agent-generated code (e.g. judging hackathon submissions, or agentic code-execution) demands isolation beyond a standard container, which shares the host kernel directly.

gVisor

gVisor is a sandboxed container runtime that inserts a user-space kernel — the **Sentry** — between the container and the host. The Sentry intercepts syscalls and reimplements much of the Linux interface in user space; only a small restricted set reaches the host. **Gofer** handles filesystem access so the sandboxed process never touches the host filesystem directly. The result is a much smaller attack surface, at a syscall/IO performance cost — a security-for-speed tradeoff reserved for untrusted, multi-tenant, or agentic workloads.

HOW IT WIRES INTO KUBERNETES

gVisor is integrated via the `runsc` runtime through containerd and selected per-pod using a **RuntimeClass** — so untrusted workloads run sandboxed while trusted ones keep normal `runc` .

Agent Sandbox & Agent Substrate

GKE Agent Sandbox (now GA) provides secure execution for agent workloads. **Agent Substrate** is a Google open-source project layered on Kubernetes for dense, low-latency scheduling of many mostly-idle agents — mapping a large set of “actors” onto a small set of warm “worker” pods, taking the control plane out of the critical path. It targets the chatter of millions of sub-second tool calls that would overwhelm a standard control plane. (Early development; not production-ready; APIs expected to change.)

9 Core Concept Review & Exam Questions

Quick-reference answers to the assessment questions covered in this training.

QUESTION	ANSWER
IAM hierarchy for a new application	Create a new folder inside your organization and create projects inside that folder. (Org → Folders → Projects → Resources.)
Manage cost for a product/customer	Set up budgets and alerts at the project level.
Managed compute with native container support	GKE clusters.
Install locally to run automation scripts	The Cloud SDK (gcloud CLI).
Problems containers solve	Isolate dependencies; fix laptop-vs-production drift; avoid VM packaging waste.
Why union file systems	Encapsulate app + dependencies into clean, minimal layers.
Significance of the top layer	Only writable layer; ephemeral — lost when the container is deleted.
Management approach Kubernetes uses	Declarative configuration.
Computers that run workloads	Nodes.
Kubernetes vs. GKE	GKE is Kubernetes as a managed service.
Scales the cluster as a whole	GKE (cluster autoscaling).
kubectl connects to which component	kube-apiserver.
Controller for persistent storage	StatefulSet.
Purpose of a Service (2)	Load-balancing network endpoint for Pods; choose how Pods are exposed.
Co-locate containers for lowest latency	Place them in the same Pod.
Deploy multiple load-balanced copies	Create a Deployment manifest with the replica count.
Isolate prod from test/staging	Namespaces per environment + ResourceQuotas on test/staging.
Persistent storage that survives Pod loss	Volumes on network-based storage , remote to the Pod.
Regional cluster, 4 nodes, default zones — VM count	12 (4×3 zones); control plane billed via management fee.
	GKE.

QUESTION	ANSWER
Deploying containers, avoid cluster mgmt overhead, max control	
Stateful applications	Apps that persist user/session state (game progress, shopping cart).
Lightweight portable packaging	Containers.
Compute Engine fine-grained cost control (2)	Per-second billing; fully customizable VMs.
Role of the kubelet	Kubernetes' agent on each node.

10 Appendix: Study URLs by Topic

Official Google Cloud and Kubernetes references, grouped by topic for focused study.

GKE Foundations & Getting Started

Google Kubernetes Engine product page

<https://cloud.google.com/kubernetes-engine>

Quickstart: Create a cluster and deploy a workload

<https://docs.cloud.google.com/kubernetes-engine/docs/quickstarts/create-cluster>

Configuring cluster access for kubectl

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/cluster-access-for-kubectl>

Access Google Cloud APIs from GKE workloads

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/workload-identity>

GKE best practices documentation

<https://docs.cloud.google.com/kubernetes-engine/docs/best-practices>

GKE security documentation hub

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/security-overview>

Kubernetes Reference & APIs

Kubernetes documentation

<http://kubernetes.io/docs/>

kubectl commands reference

<https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands>

gcloud container commands

<https://docs.cloud.google.com/sdk/gcloud/reference/container>

GKE REST API

<https://docs.cloud.google.com/kubernetes-engine/docs/reference/rest>

GKE API permissions

<https://docs.cloud.google.com/kubernetes-engine/docs/reference/api-permissions>

GKE API organization and structure

<https://docs.cloud.google.com/kubernetes-engine/docs/reference/api-organization>

Cluster Networking

Cluster Networking (Kubernetes)

<https://kubernetes.io/docs/concepts/cluster-administration/networking/>

Services, Load Balancing, and Networking

<https://kubernetes.io/docs/concepts/services-networking/>

Learn GKE networking architecture

<https://docs.cloud.google.com/kubernetes-engine/docs/learn/learn-gke-net-architecture>

Service

<https://kubernetes.io/docs/concepts/services-networking/service/>

Ingress

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

Gateway API

<https://kubernetes.io/docs/concepts/services-networking/gateway/>

About Service load balancing in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/service-load-balancer>

GKE Ingress concepts

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/ingress>

GKE networking documentation

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/network-overview>

DNS, Network Policy & IP Design**DNS for Services and Pods**

<https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>

Configure DNS for GKE clusters

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/kube-dns>

Network Policies (Kubernetes)

<https://kubernetes.io/docs/concepts/services-networking/network-policies/>

Declare Network Policy

<https://kubernetes.io/docs/tasks/administer-cluster/declare-network-policy/>

GKE network policy overview

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/network-policy>

About cluster configuration choices in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/configuration-overview>

Best practices for GKE networking

<https://docs.cloud.google.com/kubernetes-engine/docs/best-practices/networking>

Alias IP ranges for GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/alias-ips>

Dual-stack networking

<https://kubernetes.io/docs/concepts/services-networking/dual-stack/>

Topology Aware Routing

<https://kubernetes.io/docs/concepts/services-networking/topology-aware-routing/>

About Dataplane V2 in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/dataplane-v2>

AI-Augmented GKE Operations

Use the GKE remote MCP server

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/use-gke-mcp>

Official MCP support for Google services (announcement)

<https://cloud.google.com/blog/products/ai-machine-learning/announcing-official-mcp-support-for-google-services/>

Gemini CLI for Google Cloud

<https://cloud.google.com/gemini/docs/codeassist/gemini-cli>

GKE and Gemini CLI work better together

<https://cloud.google.com/blog/products/containers-kubernetes/gke-and-gemini-cli-work-better-together>

kubectl-ai official repository

<https://github.com/GoogleCloudPlatform/kubectl-ai>

Google Cloud with Gemini overview

<https://cloud.google.com/products/gemini>

Secure Agentic Execution (Sandbox / gVisor)

Agent Sandbox on GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/agent-sandbox>

GKE Sandbox overview

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/sandbox-pods>

Harden workload isolation with GKE Sandbox

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/sandbox-pods>

Agent Sandbox recap (Google Cloud Blog)

<https://cloud.google.com/blog/topics/developers-practitioners/agent-factory-recap-supercharging-agents-on-gke-with-agent-sandbox-and-pod-snapshots>

AI/ML Workloads on GKE

AI/ML workloads on GKE overview

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/machine-learning>

AI/ML orchestration on GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/integrations/ai-infra>

Core concept: About GPUs in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/concepts/gpus>

Core skill: Use GPUs in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/gpus>

Serve Gemma open models with vLLM (GPUs)

<https://docs.cloud.google.com/kubernetes-engine/docs/tutorials/serve-gemma-gpu-vllm>

Serve Gemma open models with Hugging Face TGI

<https://docs.cloud.google.com/kubernetes-engine/docs/tutorials/serve-gemma-gpu-tgi>

Serve an LLM with multiple GPUs in GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/tutorials/serve-multiple-gpu>

Deploy GPUs for batch workloads (Dynamic Workload Scheduler)

<https://docs.cloud.google.com/kubernetes-engine/docs/how-to/provisioningrequest>

About Ray on GKE

<https://docs.cloud.google.com/kubernetes-engine/docs/add-on/ray-on-gke/concepts/overview>

Agentic Workflows & ADK

Agent Development Kit docs

<https://google.github.io/adk-docs/>

ADK agents guide

<https://google.github.io/adk-docs/agents/>

Generative AI on Google Cloud

<https://cloud.google.com/ai/generative-ai>

Google Data Cloud & Analytics

Google Cloud Data Cloud overview

<https://cloud.google.com/data-cloud>

BigQuery product page

<https://cloud.google.com/bigquery>

BigQuery overview documentation

<https://docs.cloud.google.com/bigquery/docs/introduction>

BigQuery data & AI platform overview

<https://cloud.google.com/products/big-data>

Dataflow product page

<https://cloud.google.com/dataflow>

Dataflow overview documentation

<https://cloud.google.com/dataflow/docs/overview>

BigLake overview

<https://cloud.google.com/biglake/docs/introduction>

Dataplex Universal Catalog

<https://cloud.google.com/dataplex>

Multimodal AI & Vector Search

BigQuery vector search introduction

<https://cloud.google.com/bigquery/docs/vector-search-intro>

Manage vector indexes in BigQuery

<https://docs.cloud.google.com/bigquery/docs/vector-index>

Search embeddings with vector search tutorial

<https://docs.cloud.google.com/bigquery/docs/vector-search>